



**REPÚBLICA BOLIVARIANA DE VENEZUELA
UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN GENERAL DE PREGRADO
PROYECTO DE CARRERA: ingeniería informática
UNIDAD CURRICULAR: Lenguajes y Compiladores**

Teoría de decompiladores aplicados a firmware IoT y análisis de malware

NOMBRE DEL DOCENTE: Félix Márquez

NOMBRE DE ESTUDIANTES:

José Silva, V-30.810.283 SEC: 2

Nicole Serano, V-31.275.411 SEC: 2

Eloismary Díaz, V-28.682.879 SEC: 2

José Moreno, V-28.086.251 SEC: 2

CIUDAD GUAYANA, 12– 05 – 2026

Introducción

En un ecosistema tecnológico donde los dispositivos de Internet de las Cosas (IoT) y los sistemas críticos operan bajo arquitecturas cerradas, la ingeniería inversa se ha consolidado como la herramienta de defensa definitiva. Ante la ausencia de código fuente, la capacidad de diseccionar binarios para comprender su comportamiento no es solo una ventaja competitiva, sino una necesidad de seguridad nacional y corporativa.

Sin embargo, el análisis de software compilado enfrenta hoy una "tormenta perfecta": por un lado, la sofisticación del malware moderno y sus técnicas de ofuscación; por otro, las limitaciones intrínsecas de los decompiladores actuales, cuya precisión semántica aún dista de ser ideal. El presente documento explora los pilares de la ingeniería inversa contemporánea, desde la evolución de las arquitecturas de descompilación y el análisis de flujo de datos, hasta las estrategias avanzadas para la desofuscación de malware e identificación proactiva de vulnerabilidades. A través de este recorrido, analizaremos cómo la integración de modelos de lenguaje (LLMs) y técnicas híbridas está transformando el análisis de binarios en una disciplina cada vez más automatizada y precisa.

1. Ingeniería Inversa de Decompiladores

La ingeniería inversa de binarios constituye un pilar fundamental en el análisis de sistemas críticos, desde firmware de dispositivos IoT hasta malware sofisticado, donde la ausencia de código fuente hace indispensable el uso de decompiladores para recuperar representaciones de alto nivel del programa (Cao et al., 2024). Sin embargo, la efectividad de estas herramientas se ve limitada por su incapacidad para manejar binarios no estándar, los cuales son cada vez más comunes en entornos embebidos y sistemas industriales, lo que obliga a los analistas a invertir una cantidad significativa de tiempo en la comprensión manual de formatos propietarios (Lee et al., 2025). Esta problemática ha impulsado el desarrollo de nuevas técnicas que trascienden el análisis estático tradicional.

Arquitectura de Descompilación Moderna

Representación Intermedia (IR) y Elevación de Código: Los decompiladores contemporáneos, como Ghidra y Hex-Rays, traducen inicialmente el código máquina a una representación intermedia independiente de la arquitectura (P-Code en Ghidra, microcódigo en Hex-Rays), lo que permite aplicar transformaciones semánticas sin depender de instrucciones específicas del procesador (Cao et al., 2024). Esta capa de abstracción resulta esencial para normalizar variaciones sintácticas introducidas por diferentes compiladores (Wodiany et al., 2024).

Limitaciones de Exactitud Semántica: Contrario a la creencia generalizada en la comunidad de ingeniería inversa, las evaluaciones sistemáticas recientes revelan que incluso el decompilador comercial más avanzado, Hex-Rays, alcanza apenas un 55% de precisión en la recuperación semántica del código original (Cao et al., 2024). Esta brecha de efectividad explica por qué muchos analistas aún prefieren trabajar directamente con código ensamblador en tareas complejas de análisis.

Estrategias de Mejora y Automatización

Enfoques Híbridos Estático-Dinámicos: Herramientas como LeanBin combinan el levantamiento de código (lifting) estático sin heurísticas con trazas de ejecución dinámica para reconstruir el flujo de control real del programa, logrando reducciones significativas en la superficie de ataque del binario (hasta un 53.59% del tamaño original) mientras mantienen un overhead de ejecución de solo 1.21× (Wodiany et al., 2024).

Integración de Modelos de Lenguaje de Gran Escala (LLMs): Estudios recientes han demostrado que la integración de LLMs en flujos de trabajo de decompilación, mediante plugins como GhidraMCP asistido por Claude Sonnet 4, puede acelerar la identificación de rutinas críticas (cifrado, comunicación C2) y renombrar automáticamente variables y funciones ofuscadas (Gorzynski & Donaldson, 2025). No obstante, estas aproximaciones aún presentan limitaciones como la alucinación de anotaciones incorrectas y la incapacidad para realizar parcheo binario dinámico.

Validación Mediante Recompilación: Una estrategia prometedora para evaluar y mejorar la fidelidad de los decompiladores consiste en aplicar un enfoque de especulación-verificación-corrección, donde el código descompilado se recompila y se compara con el binario original, refinando iterativamente las partes divergentes (Cao et al., 2024). Este método ha demostrado ser más efectivo que la simple expansión de reglas heurísticas.

La evolución apunta hacia la combinación sinérgica de análisis semántico robusto, técnicas de aprendizaje automático y validación dinámica, con el objetivo de cerrar progresivamente la brecha entre la precisión actual (~55%) y un ideal de fidelidad semántica que permita equiparar el análisis de binarios al de código fuente disponible.

2. Análisis de Flujo de Datos

El análisis de flujo de datos constituye una técnica estática fundamental para comprender cómo se propagan los valores a través de las variables de un programa sin necesidad de ejecutarlo, permitiendo detectar vulnerabilidades potenciales, rastrear entradas contaminadas (taint tracking) y optimizar el código mediante la eliminación de operaciones redundantes (Andrade et al., 2020). Su aplicación en binarios, sin embargo, se enfrenta a desafíos adicionales derivados de la pérdida de información de tipos y la complejidad de identificar relaciones entre variables en un entorno sin metadatos.

Flujo de Datos Intraprocedural

Cadenas Definición-Uso (Def-Use Chains): La técnica central del análisis de flujo de datos consiste en construir cadenas que conectan cada definición de una variable (el punto donde se le asigna un valor) con todos sus usos posteriores, permitiendo evaluar si una variable puede ser utilizada antes de ser inicializada o si existen asignaciones redundantes (Feng, 2026). En el contexto de binarios, la recuperación precisa de estas cadenas requiere identificar convenciones de llamada y eliminar efectos secundarios de instrucciones complejas.

Análisis de Alias y Punteros: Uno de los mayores desafíos en el análisis de flujo de datos es la correcta identificación de cuándo dos referencias apuntan a la misma ubicación de memoria (aliasing). Los binarios ofuscados o generados por compiladores agresivos pueden introducir niveles adicionales de indirección que oscurecen las relaciones reales entre variables (Andrade et al., 2020).

Flujo de Datos Interprocedural

Program Slicing y Dependencias entre Funciones: El análisis interprocedural

extiende las cadenas definición-uso a través de los límites de funciones, rastreando cómo los parámetros de entrada afectan los valores de retorno y las salidas. Técnicas como el program slicing permiten aislar las instrucciones relevantes para un comportamiento específico, reduciendo la complejidad del análisis en programas extensos (Feng, 2026). Herramientas como angr implementan estas técnicas para guiar procesos de fuzzing más precisos.

Recuperación de Tipos mediante Few-Shot Learning: La ausencia de información de tipos en binarios stripped (sin símbolos) ha sido tradicionalmente una barrera para el análisis de flujo de datos preciso. Frameworks recientes como TypeFSL combinan el análisis de flujo de datos interprocedural con técnicas de few-shot learning, logando predecir tipos complejos con una precisión promedio del 77.9% en escenarios de 20-way 5-shot, superando significativamente a los enfoques basados únicamente en reglas heurísticas (Shen et al., 2024).

Aplicaciones en Seguridad

Detección de Vulnerabilidades: El análisis de flujo de datos permite identificar patrones vulnerables, como el uso de funciones peligrosas (strcpy sin verificación de límites) o la propagación de entrada no validada a zonas sensibles (command injection). La integración con motores de análisis simbólico facilita la generación automática de casos de prueba que ejercitan rutas críticas de ejecución (Feng, 2026).

Desofuscación y Normalización: Las técnicas de ofuscación que alteran artificialmente el flujo de datos (inserción de código muerto, predicados opacos, expansión de bucles) pueden ser contrarrestadas mediante análisis de flujo de datos que identifican dependencias reales e ignoran operaciones que no afectan el estado observable del

programa (Huici et al., 2026).

El análisis de flujo de datos continúa evolucionando hacia enfoques híbridos que combinan técnicas estáticas con ejecución simbólica y aprendizaje automático, permitiendo escalar a programas de gran tamaño sin sacrificar precisión en la detección de vulnerabilidades complejas.

3. Desofuscación y Análisis de Malware

Actualmente de la ciberseguridad en dispositivos de Internet de las Cosas (IoT) revela una sofisticación creciente en el malware, el cual emplea técnicas avanzadas de ofuscación y empaquetado para evadir la detección tradicional (Matrosov et al., 2019). La desofuscación efectiva requiere ahora trascender el análisis sintáctico superficial, adoptando métodos de análisis semántico basados en Lenguajes de Representación Intermedia (IR), como HLIL o VEX, para normalizar el código y recuperar su lógica intrínseca (Huici et al., 2026).

Esto es debido a un problema donde el análisis de malware moderno se enfrenta al desafío de la "protección de software", donde el código es deliberadamente endurecido para resistir la ingeniería inversa mediante la alteración de su estructura y flujo mediante diferentes técnicas. Algunas de esas técnicas más comunes serían:

Métodos de ofuscación

- **Empaquetado (Packing):** Los autores de malware utilizan empaquetadores para comprimir o cifrar el ejecutable original, moviendo o protegiendo el punto de entrada original (OEP) para frustrar el análisis estático y evadir motores de antivirus

- Virtualización de Código: Es una de las técnicas más complejas; consiste en convertir el código original en un "bytecode" (un lenguaje intermedio) que solo puede ser ejecutado por un intérprete, en este caso una máquina virtual personalizada incluido en el malware.
- Flattening (Aplanamiento de Grafo): Transforma la estructura lógica de un programa en un solo bucle central transformando en un despachador que dirige la ejecución del malware, eliminando las pistas visuales sobre el flujo original del código.

Análisis Semántico y Normalización

- Análisis Semántico y Normalización: Para contrarrestar la ofuscación, se eleva el código máquina a niveles de abstracción estables como HLIL (High-Level Intermediate Language) (Huici et al., 2026). Esto permite ignorar detalles arquitectónicos específicos y centrarse en la lógica de las variables y el flujo de control (Dang et al., 2014).
En lugar de buscar patrones de texto simples, el análisis semántico busca entender la lógica del código elevándolo a Lenguajes Intermedios para su posterior análisis
- Análisis de Handlers y VMS: En casos de virtualización de código, donde el malware ejecuta su propio bytecode, el análisis semántico se enfoca en identificar la función de transferencia de cada "handler" para reconstruir el algoritmo original a partir del intérprete (Dang et al., 2014).
- Ejecución Simbólica y Concoholica: Estas técnicas permiten explorar múltiples rutas de ejecución mediante la resolución de restricciones lógicas, ayudando a determinar

los valores necesarios para activar ramas de código específicas o desofuscar predicados opacos (Dang et al., 2014).

4. Identificación de Vulnerabilidades

La identificación proactiva de fallos mediante Análisis Estático (SAST) en el firmware de dispositivos integrados se ha consolidado como una herramienta fundamental para mitigar riesgos antes del despliegue masivo en infraestructuras críticas (Nuray et al., 2026). Permitiendo la correlación de vulnerabilidades conocidas (N-days) mediante resúmenes de similitud como TLSH (Huici et al., 2026). Por ende conocer el funcionamiento del SAST es relevante.

Análisis Estático (SAST)

El SAST consiste en la inspección de binarios, scripts y librerías sin ejecutar el código, permitiendo detectar debilidades estructurales, configuraciones inseguras y credenciales embebidas (Nuray et al., 2026). Integrando automatizaciones con frameworks como EMBA que integra múltiples módulos de análisis estático (como Sengrep y Radare2) para identificar componentes desactualizados y verificar protecciones binarias, generando reportes estructurados para los profesionales (Nuray et al., 2026). Aunque eficiente, el análisis estático a menudo produce falsos positivos al no poder observar el comportamiento dinámico o las interacciones con el hardware real, lo que requiere ser complementado con análisis dinámico o emulación (Nuray et al., 2026).

Triage de Vulnerabilidades mediante Similitud

El uso de resúmenes de similitud (similarity digests) mediante algoritmos como TLSH (Locality Sensitive Hashing) permite generar huellas digitales de funciones que son resistentes a pequeñas variaciones sintácticas causadas por diferentes compiladores u

optimizaciones (Huici et al., 2026), lo cual facilita la correlación de N-days a través de sistemas como APOTHEOSIS, permitiendo realizar búsquedas de "vecinos más cercanos" para identificar si un patrón de código vulnerable conocido (CVE) está presente en miles de imágenes de firmware de forma casi instantánea y sin depender de los metadatos de los proveedores (Huici et al., 2026). Asimismo, mediante el análisis de flujo de datos (def-use chains) y motores de análisis binario como angr, es posible extraer cadenas de definición-uso para rastrear cómo las entradas afectan el estado del programa, sirviendo como una guía fundamental para procesos de pruebas dinámicas (fuzzing) mucho más precisos (Feng, 2026).

Conclusión

La brecha entre el análisis de código fuente y el análisis de binarios se está cerrando, aunque no sin desafíos técnicos significativos. Como se ha expuesto, la efectividad de la ingeniería inversa moderna ya no depende exclusivamente de la lectura estática de instrucciones, sino de una convergencia estratégica de métodos: la abstracción mediante lenguajes intermedios (IR), la potencia predictiva del machine learning para la recuperación de tipos y el rigor de la validación mediante recompilación.

En última instancia, el éxito en la detección de vulnerabilidades y la neutralización de malware reside en la capacidad de los analistas para adoptar enfoques híbridos. Mientras que el análisis estático (SAST) y los resúmenes de similitud (como TLSH) permiten procesar volúmenes masivos de datos a gran escala, la ejecución simbólica y el análisis semántico profundo aportan la precisión necesaria para dismantelar capas complejas de ofuscación. El futuro de la disciplina apunta hacia una simbiosis donde la inteligencia artificial asista en las tareas repetitivas de renombrado y triaje, permitiendo que el experto humano se concentre en la lógica crítica y la resolución de problemas donde la semántica y el contexto siguen siendo los reyes.

Referencias Bibliográficas

- Dang, B., Gazet, A., & Bachaalany, E. (2014). Practical Reverse Engineering: x86, x64, ARM, Windows Kernel, Reversing Tools, and Obfuscation. John Wiley & Sons.
- Feng, K. (2026). Fuzzing techniques for automated vulnerability detection in IoT firmware (Tesis doctoral). University of Glasgow.
- Huici, D., Yousefnezhad, N., Rodríguez, R. J., & Costin, A. (2026). Automating firmware vulnerability triage via high-level representations and similarity digests. En Workshop on Binary Analysis Research (BAR) 2026.
- Maghanaki, M., Keramati, S., Chen, F. F., & Shahin, M. (2026). Systematic Evaluation of Machine Learning and Deep Learning Models for IoT Malware Detection Across Ransomware, Rootkit, Spyware, Trojan, Botnet, Worm, Virus, and Keylogger. Sensors, 26(6), 1750. <https://doi.org/10.3390/s26061750>
- Matrosov, A., Rodionov, E., & Bratus, S. (2019). Rootkits and Bootkits: Reversing Modern Malware and Next-Generation Threats. No Starch Press.
- Nuray, K. S., Upton, O., & Beebe, N. L. (2026). Comparing the Use of EMBA for IoT Firmware Security Analysis on Cloud Services and Standalone Servers. Journal of Cybersecurity and Privacy